

ArgParser

Tereza Jandová
Adam Janeček

Ukázka deklarace:

- Deklarativní design
- Využití reflexe

```
[ExampleUsage("myProgram [options]")]  
class SimpleArgs : BaseArgs  
{  
    [ShortOptions('i'), LongOptions("int")]  
    public int? IntOption { get; set; }  
  
    [ShortOptions('s', 't'), Required]  
    public string StrOption { get; set; }  
  
    [ShortOptions('e'), LongOptions("enum")]  
    public MyEnum En { get; set; } = MyEnum.A;  
  
    public override string[] PlainArguments  
        { get; set; } = [];  
}
```

Použití:

```
private static void Main(string[] args) {  
    ArgParser<SimpleArgs> simpleParser = ArgParserFactory.FromType<SimpleArgs>();  
    try {  
        SimpleArgs simpleArgs = simpleParser.Parse(args);  
        if (simpleArgs.IntOption is int intVal)  
            // Do desired functionality with the given value  
    } catch (CommandLineParsingException ex) {  
        Console.WriteLine(ex.Message);  
    } catch (HelpCalledException helpEx) {  
        Console.WriteLine(simpleParser.GenerateHelpMessage());  
    }  
}
```

Podporujeme:

- Jakýkoliv typ argumentu implementující IParsable
- Vlastní validátory na Property i Class
- Terminating flag (například Help, Version,...)
- Analyzátory Roslyn validující použití atributů

Nepodporujeme:

- Větší počet argumentů u options než 1
- Striktní pozice argumentů
- Subcommands

BaseArgs:

- Obsahuje PlainArguments a HelpCalled
- Předek pro uživatelsky definované třídy argumentů

```
public abstract class BaseArgs
{
    [ShortOptions('h')]
    [LongOptions("help")]
    [TerminatingFlag<HelpCalledException>]
    public virtual bool HelpCalled
        { get; set; }

    public abstract string[] PlainArguments
        { get; set; }
}
```

Atributy:

- Definují vlastnosti
- Použitelné na Class a Property

▼ Attribute

▶  ArgParser.Attributes.ClassValidatorAttribute< TArgs >

 ArgParser.Attributes.EnumCasePolicyAttribute

 ArgParser.Attributes.ExampleUsageAttribute

 ArgParser.Attributes.HelpAttribute

 ArgParser.Attributes.LongOptionsAttribute

▶  ArgParser.Attributes.OptionValidatorAttribute< TType >

 ArgParser.Attributes.RequiresAttribute

 ArgParser.Attributes.ShortOptionsAttribute

 ArgParser.Attributes.TerminatingFlagAttribute< TException >

 ArgParser.Attributes.ValuePlaceholderAttribute

ShortOptionsAttribute

- Definuje krátké jména
- Použití s jednou pomlčkou (-)

```
internal sealed class Args : BaseArgs
{
    [ShortOptions('i', 'o')]
    public int? IntOption { get; set;}
}
```

LongOptionsAttribute

- Definuje dlouhá jména
- Použití se dvěma pomlčkami (--)

```
internal sealed class Args : BaseArgs
{
    [LongOptions("int", "intopt")]
    public int? IntOption { get; set;}
}
```


EnumCasePolicyAttribute

- Definuje formát argumentu

Možnosti:

- PreserveCase (Default)
- AllLowerCase
- AllUpperCase

```
[EnumCasePolicy(EnumCase.PreserveCase)]
internal enum MyEnum
{
    First,
    Second,
    Third,
}

internal sealed class Args : BaseArgs
{
    [ShortOptions('e')]
    public MyEnum Enum { get; set; };
}
```

RequiresAttribute

- Definuje závislost optionu na přítomnosti jiného optionu

```
internal sealed class Args : BaseArgs
{
    [ShortOptions('i')]
    public int? IntOption { get; set; }

    [ShortOptions('s')]
    [Requires(nameof(IntOption))]
    public string? StringOption { get; set; }
}
```

TerminatingFlagAttribute<TException>

- Alternativní průběh programu
- Umožňuje předčasné ukončení parsování a přeskočení validací
- Uživatelem definovaná výjimka

```
internal sealed class Args : BaseArgs
{
    [ShortOptions('f')]
    [TerminatingFlag<FlagCalledException>]
    public bool Flag{ get; set; }
}

...

try { Args a = Parser.Parse(args);}
catch (CommandLineParsingException ex) { }
catch (FlagCalledException flagEx) { }
catch (HelpCalledException helpEx) { }
```

ClassValidatorAttribute<TArgs>

- Umožňuje vlastní validace argumentových tříd
- Předdefinovaná implementace: ``AllowPlainArguments(bool)``

```
[AllowPlainArguments(false)]  
[CustomClassValidator]  
internal sealed class Args : BaseArgs { }  
  
internal sealed class CustomClassValidator :  
    ClassValidatorAttribute<Args>  
{  
    public override bool Validate(Args args,  
        out string? errorMessage) { ... }  
}
```

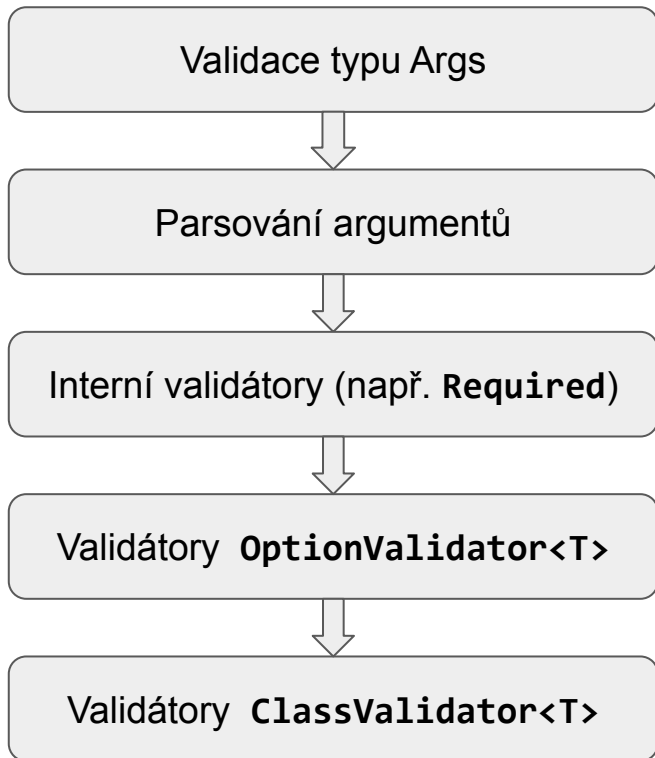
OptionValidatorAttribute<TType>

- Umožňuje vlastní validace Property
- Předdefinovaná implementace: ``Range<T>(T min, T max)``

```
internal sealed class Args : BaseArgs
{
    [Range<int>(0, int.MaxValue)]
    [CustomPropertyValidator]
    public int IntOption { get; set; }
}

internal sealed class CustomPropertyValidator :
    OptionValidatorAttribute<int>
{
    public override bool Validate(Args args,
        out string? errorMessage) { ... }
}
```

Flow graf



```
[AllowPlainArguments(false)]  
class Args : BaseArgs  
{  
    [LongOptions("int"), Required]  
    [Range<int>(1, 5)]  
    public int? IntOption { get; set; }  
  
    [ShortOptions('s')]  
    [Requires(nameof(IntOption))]  
    public string StrOption { get; set; }  
  
    [ShortOptions('V')]  
    [TerminatingFlag<VersionCalledException>]  
    public bool Version { get; set; }  
}
```

HelpAttribute, ValuePlaceholderAttribute, ExampleUsageAttribute

- Definuje help message

Příklad:

Usage: Example text

Options:

--int=INT_VALUE

Example of int option.

```
[ExampleUsage("Example text")]
internal sealed class Args : BaseArgs
{
    [LongOptions("int")]
    [Help("Example of int option.")]
    [ValuePlaceholder("INT_VALUE")]
    public int? IntOption { get; set; }
}
```

ArgParser:

- Parsování předaných argumentů

```
ArgParser<SimpleArgs> simpleParser = ArgParserFactory.FromType<SimpleArgs>();  
try {  
    SimpleArgs simpleArgs = simpleParser.Parse(args);  
    if (simpleArgs.IntOption is int intVal)  
        // Do desired functionality with the given value  
} catch (CommandLineParsingException ex) {  
    Console.WriteLine(ex.Message);  
} catch (HelpCalledException helpEx) {  
    Console.WriteLine(simpleParser.GenerateHelpMessage());  
}
```


Nevýhody:

- Implementace reflexí
- Pouze statická definice optionů
- Limitace atributů v C#

Výhody:

- Přímočarý přístup uživatele k naparsovaným hodnotám
- Nezávislost zbytku programu na ArgParseru
- Minimum funkčního kódu
- Rozšiřitelnost pomocí vlastních validátorů